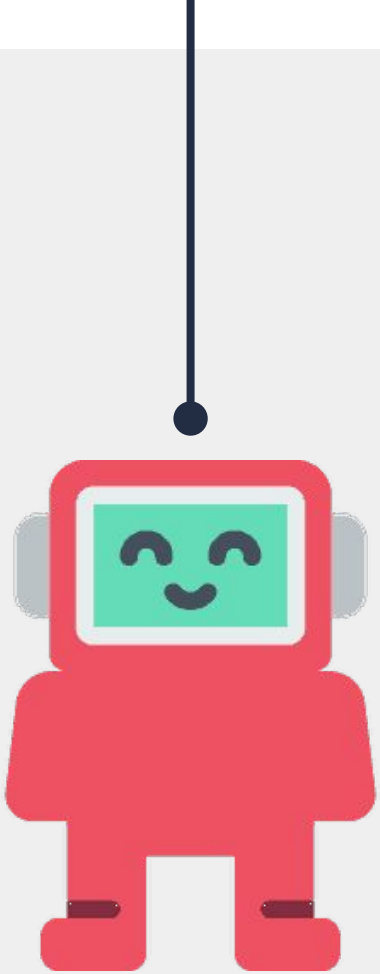
Scientific research using AI

Letting AI agents do research, without losing control

Tomáš Pavlík, 2026



Fluoroquinolones are being used as antibiotics

1. Enzyme cuts the DNA



Makes a transient double-strand cut to relieve supercoiling.

2. Drug traps the complex



The fluoroquinolone locks the cut enzyme-DNA complex; it cannot reseal.

3. DNA breaks, cell dies



Permanent double-strand breaks are lethal.

Bactericidal

Which enzyme, which bug

DNA gyrase = primary target in gram-negative bacteria.

Topoisomerase IV = primary target in gram-positive bacteria.

Memory hook: gram-Negative to Gyrase; gram-Positive to topo IV.

Key features

Bactericidal and concentration-dependent killing.

Resistance is mainly by QRDR mutations in gyrA, gyrB, parC, and parE (often stepwise).



Resistance hides in a few codons of two genes

THE DRUG

Ciprofloxacin and other fluoroquinolones trap DNA gyrase and topoisomerase IV on DNA, killing the cell.

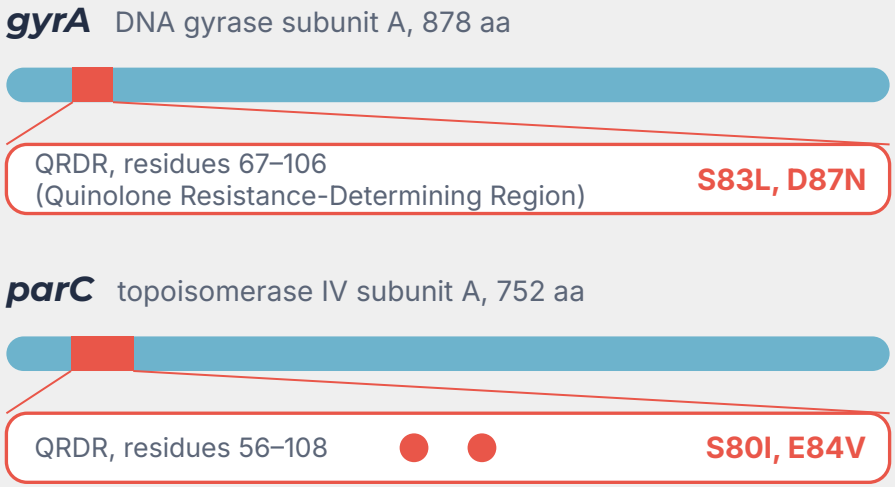
THE ESCAPE

Point mutations in a short region of *gyrA* and *parC*, the QRDR, block the drug. More mutations, more resistance.

WHY IT MATTERS

Resistant *E. coli* is common and often behind failed treatment, e.g. of urinary tract infections.

Plasmid genes (qnr) and efflux pumps also cause resistance; out of scope today.



TODAY One sample's reads in; a call out: resistant, susceptible, or inconclusive.

From raw reads to a resistance call

1 Fetch

`NCBI datasets, ENA`

→ reference FASTA + GFF, paired reads

2 QC

`fastp, seqkit`

→ trimmed reads, ~30× subsample

PBS job

3 Align

`bwa-mem2 or minimap2, samtools`

→ sorted, indexed BAM

PBS job

4 Inspect

`samtools flagstat, depth`

→ mapping rate, coverage

PBS job

5 Call

`bcftools mpileup, call`

→ VCF of variants

PBS job

6 Annotate

`bcftools, bedtools`

→ hits in the gyrA and parC QRDRs

PBS job

7 Decide

`DECISION.md rule`

→ resistant, susceptible or inconclusive

8 Render

`matplotlib or ggplot2`

→ coverage plot, variant map

PBS job

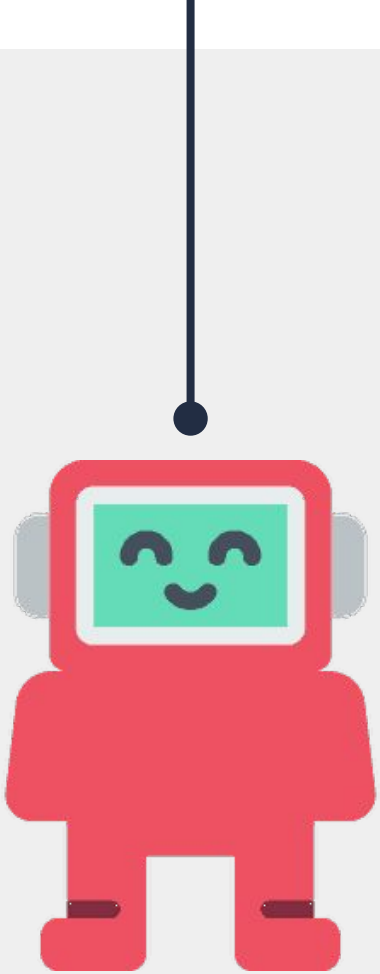
One question, one pipeline, seven concepts

The question: is this *E. coli* sample fluoroquinolone-resistant?



#1
How much autonomy?

Introduce and set up harness, connect to cluster, fetch sequences



What is autonomy?



What is autonomy?

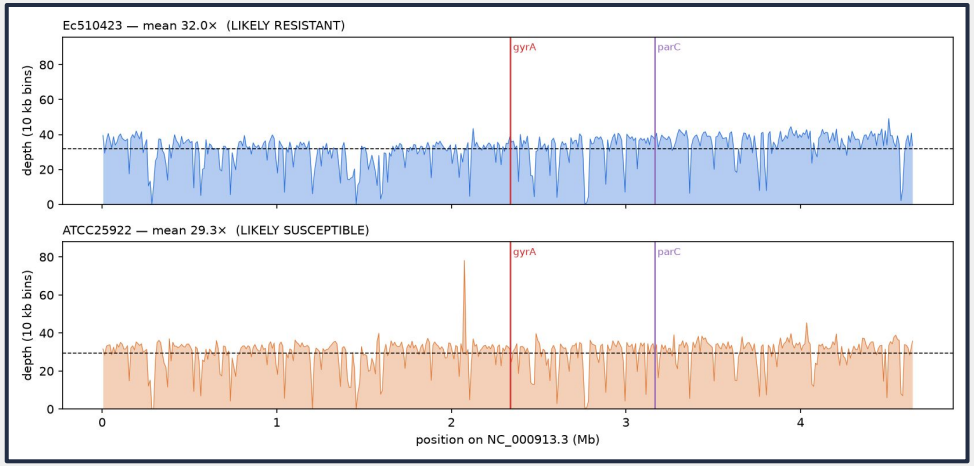


What is autonomy?

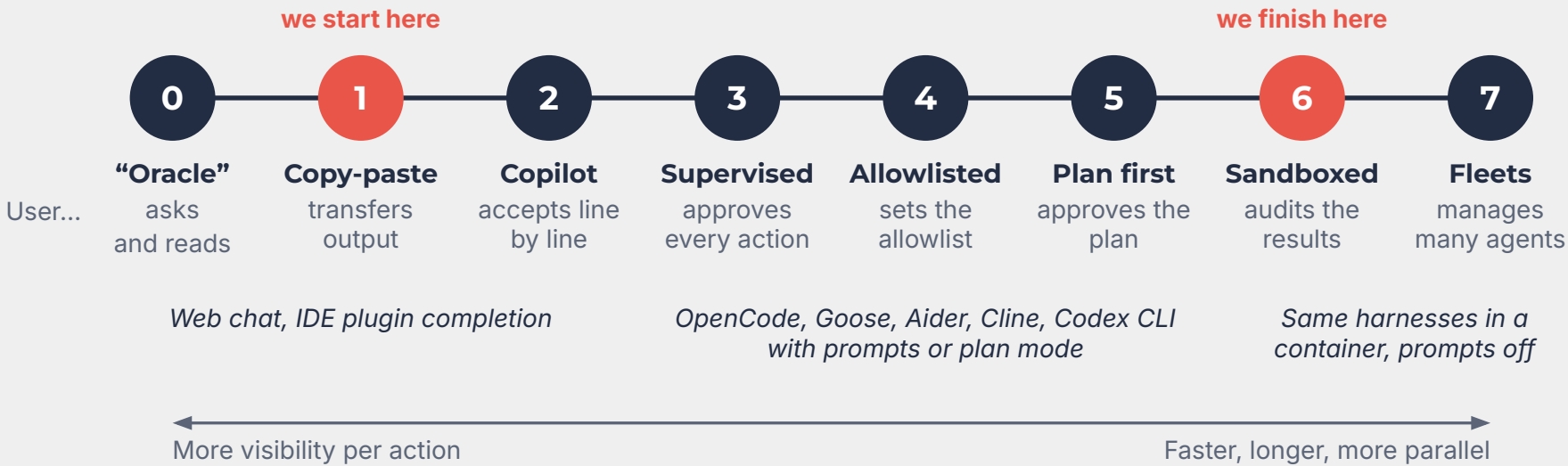
```
Create visualization of Ec510423 and ATCC25922 reads coverage on
NC_00913.3

Build · glm-5.3 LiteLLM · high

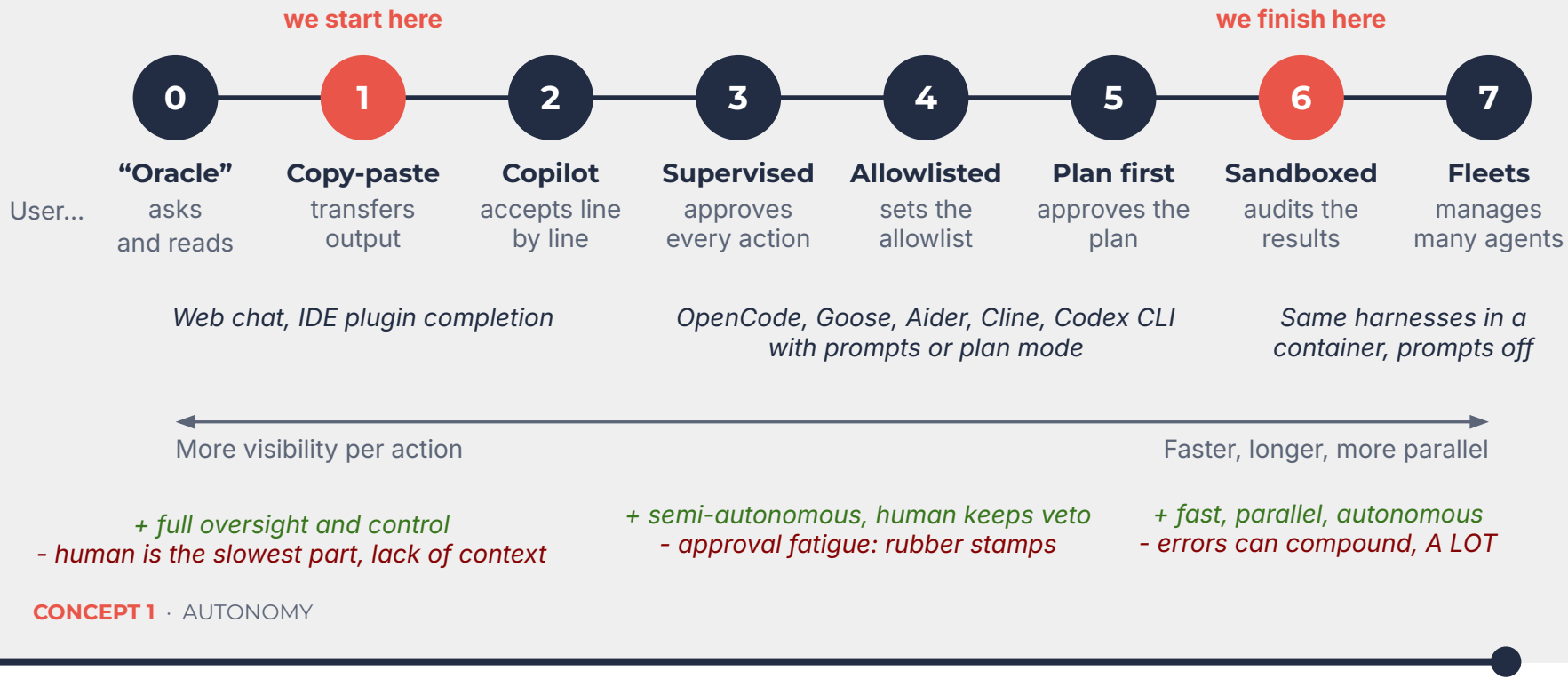
~/work/agentic_workshop_bioinf          shift+tab agents  ctrl+p commands
```



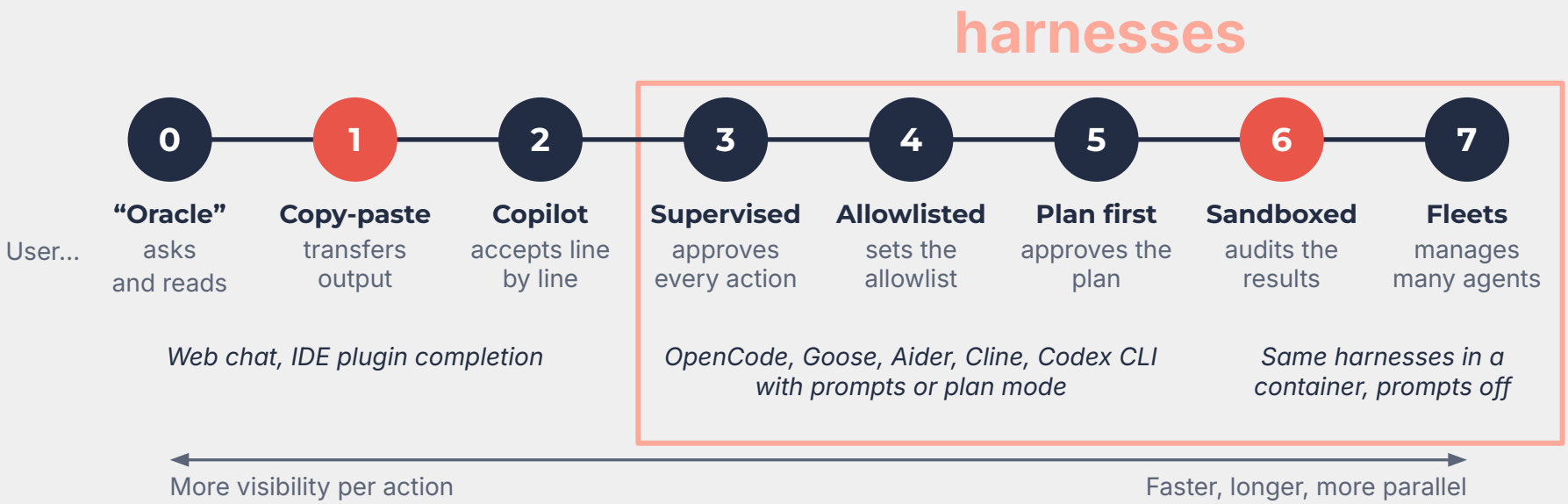
Autonomy = spectrum from doing to auditing



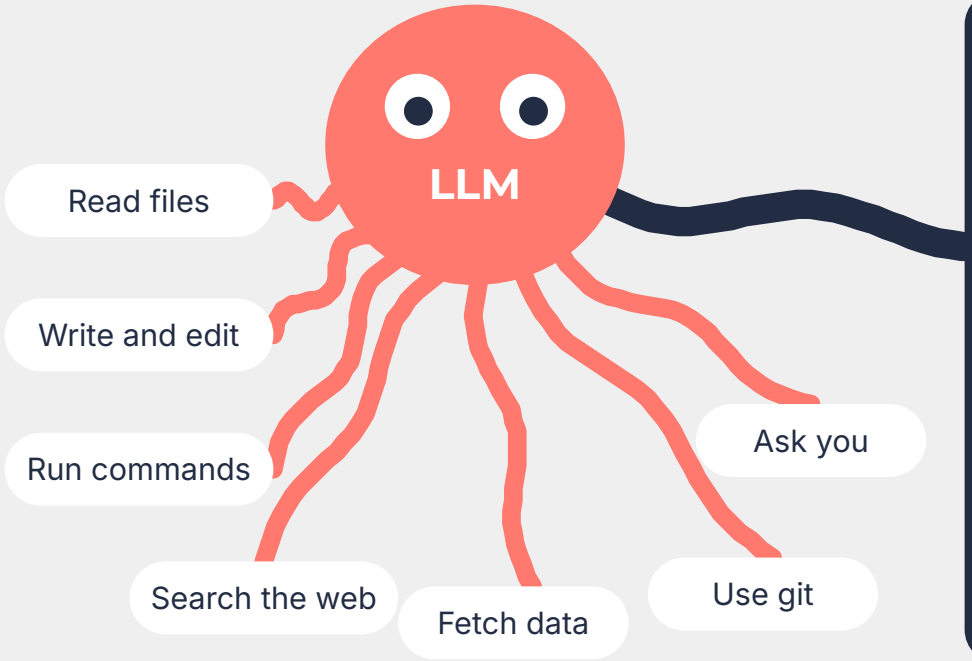
Autonomy = spectrum from doing to auditing



Autonomy = spectrum from doing to auditing



The model talks; the harness gives it arms



THE ARM THAT MATTERS

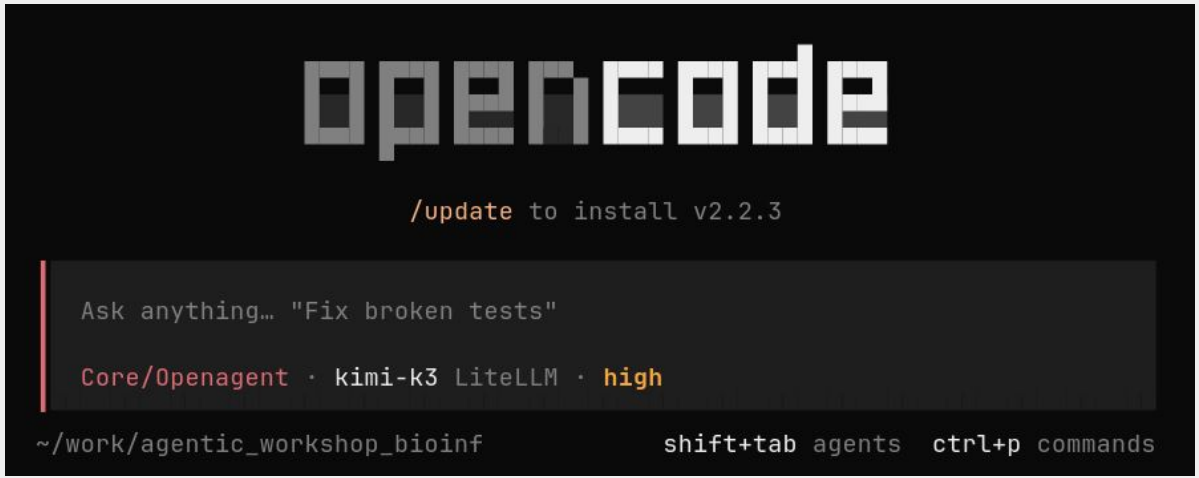
Cluster connection

- **Data:** hundreds of GBs or TBs (!!)
- **Compute:** calculations take days, many CPU/GPU-hours, lots of RAM
- **Scale:** heavy parallelization

Today's E. coli would fit on a laptop. ***Most of the projects you'll be working on won't.***

The start: Starting up the harness, play with navigation

COMMAND: OPENCODE_ENABLE_EXA=1 opencode
/model kimi-k3
/agents core/openagent



COMMAND: metacentrum-mcp login

PROMPT #1: Use the metacentrum MCP tools to test the cluster connection: call connection_status, then cluster_info. Tell me my MetaCentrum username and my working directory on the cluster. Don't search local files.

Onto cluster: Connecting the MCP server

```
+ Thought · 9.1s

I'll call both tools now — connection_status first, then cluster_info.

✓ execute
› metacentrum.connection_status
› metacentrum.cluster_info

+ Thought · 15.4s

Connection is healthy. Here's the summary:

- MetaCentrum username: tomsko (principle tomsko@META, Kerberos ticket valid for ~8 more hours)
- Cluster working directory (project workspace): /storage/praha5-elixir/projects/bioinf-fi/tomsko —
  relative paths in all jobs/tools resolve here, with 149T free of 320T

A few extras worth noting:

- Frontend: skirit.grid.cesnet.cz, connected via a persistent SSH master (no active jobs or interactive
  sessions right now)
- Home: /storage/brno2/home/tomsko (readable, but do work in the workspace)
- Default queue: elixircz@pbs-m1.metacentrum.cz (defaults: 1 CPU, 4 GB mem, 1 h walltime; caps: 16
  CPUs, 64 GB, 200 GB scratch, 5 active jobs)
- A shared workshop conda env (samtools, bcftools, bwa-mem2, fastp, seqkit, etc.) is auto-sourced at
  the start of every batch job and interactive shell.
```

Project structure: Set up remote project structure

PROMPT #2: We are starting a bacterial resequencing project on MetaCentrum:
does E. coli isolate Ec510423 carry fluoroquinolone-resistance mutations in gyrA/parC? ATCC 25922 is the susceptible control.

Using the metacentrum MCP tools, create the project directory `ecoli_fq` in my cluster workspace with this layout:
  data/ref reference genome + annotation
  data/reads raw reads
  scripts every script we run (jobs must be rerunnable without you)
  qc results logs
plus README.md (the question, samples, layout) and PROGRESS.md (a checklist of steps).
Show me the resulting tree. Do not download anything yet.

Downloading the data: Fetching reference and reads

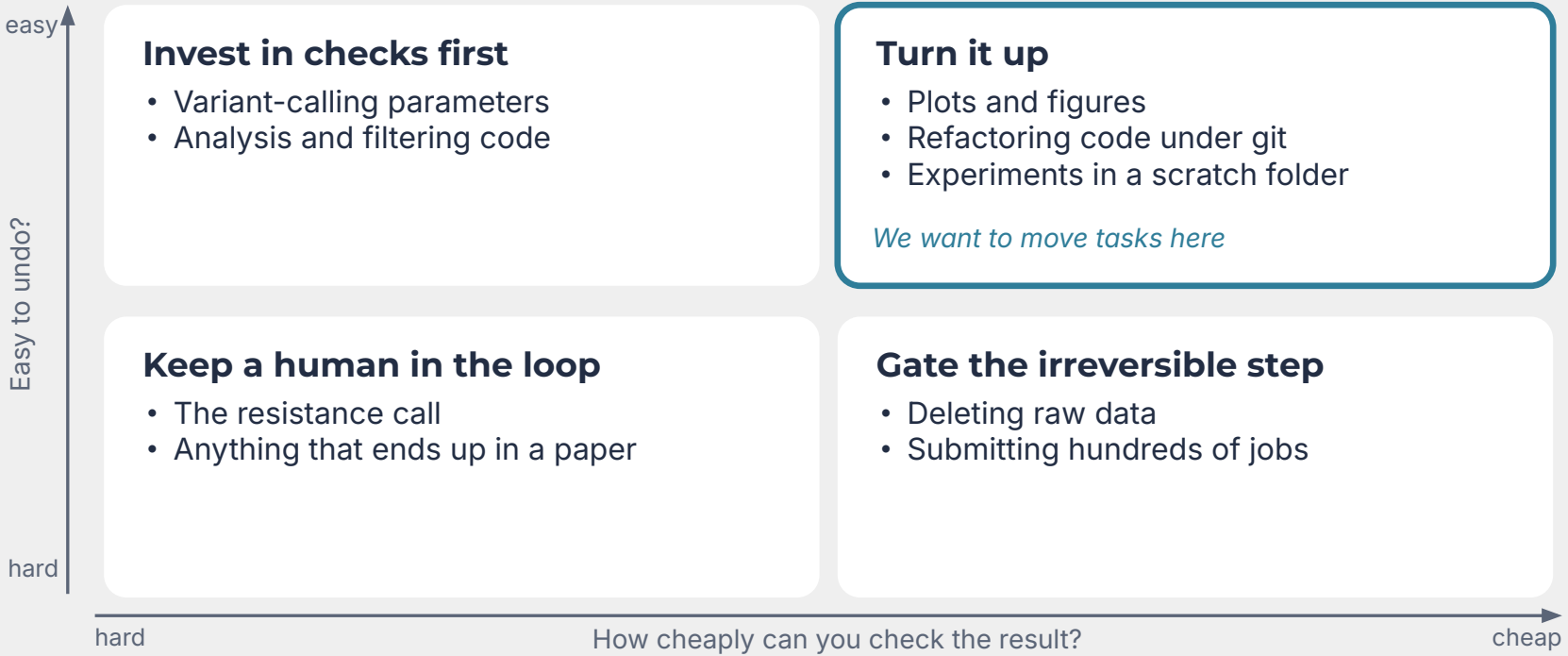
PROMPT #3: Fetch the inputs into ecoli_fq/data as a PBS batch job (not on the frontend):
- reference: E. coli K-12 MG1655, NCBI GCF_000005845.2, genomic FASTA + GFF (use datasets from the shared env)
- PE reads from ENA: Ec510423 = SRR15042476, ATCC25922 = SRR3608182 (verify FASTQ md5s against ENA)

Write scripts/fetch.sh, submit it, wait for it to finish. Requirements:
- Paths: MCP jobs start in the workspace root, so the submitted body must export PBS_O_WORKDIR=<abs path of ecoli_fq>; keep plain cd "$PBS_O_WORKDIR" in the saved script so qsub scripts/fetch.sh works from ecoli_fq/.
- Don't guess formats: inspect the datasets zip (unzip -l) before copying FASTA/GFF; look up fastq_ftp/fastq_md5 in ENA's TSV by header (accession is column 1).
- The script writes md5 report, R1/R2 read counts, and FASTA/GFF seq-ID check into logs/ and exits non-zero on any mismatch.

Then verify from logs + listing: files non-empty, md5s match, R1==R2, seq IDs equal. No trimming. Log all job ids (incl. failures) in PROGRESS.md and summarize.

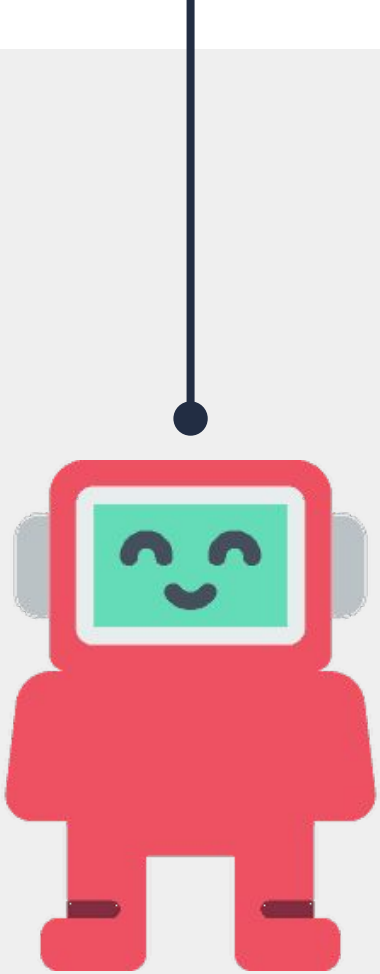
Don’t be afraid of long prompts :-)

Takeaway: autonomy is correlated with risk cost



#2
How to chose model, agent, and harness?

Try different models, show agents and skills, and run QC on our downloaded reads



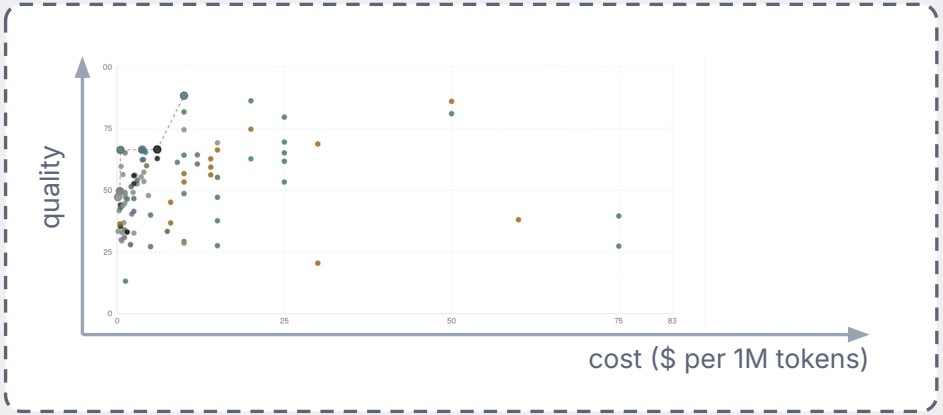
1 · Model: the part that generates

HARNESS
loop · tools · permissions · context

AGENT
instructions · skills

MODEL
predicts the next token

Predicts the next token, again and again.
Knows a lot, but can't open a file or run a command.



benchlm.ai/llm-price-performance

TRADE OFFs:

quality · speed · context window · cost · where it runs

2 · Agent: the role and its know-how



DEFINITION · who it is `agents/analyst.md`

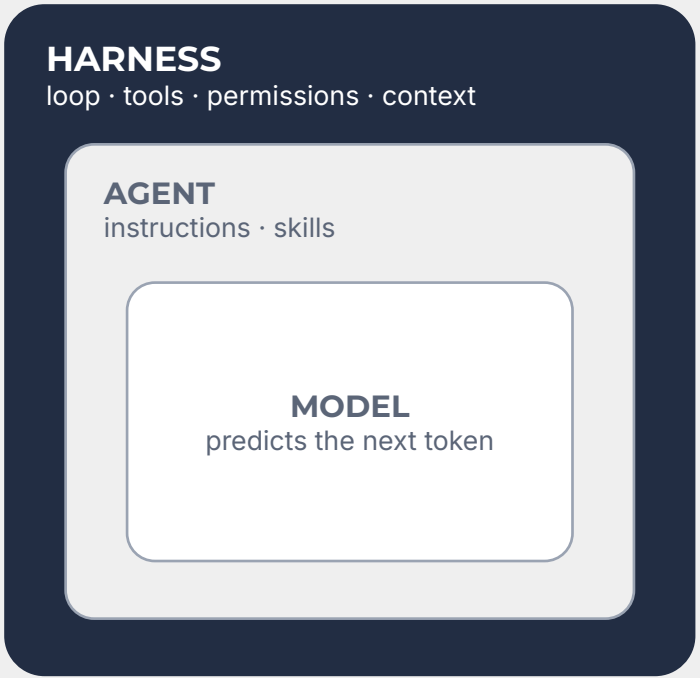
```
---
name: bioinfo-analyst
description: runs and checks variant analyses
model: <your university-hosted model>
tools: read, write, bash, grid-mcp
permissions: allow *
---
You are a careful bioinformatics analyst.
Never print raw FASTQ, SAM or VCF files.
Log every decision in NOTES.md.
```

SKILL · loaded when needed `submit-pbs-job/SKILL.md`

```
---
name: submit-pbs-job
description: use when a step needs real compute
---
1. Fill the PBS template; cap at 2 h and 8 CPUs
2. Submit; log the job ID in PROGRESS.md
```

CONCEPT 2 · MODEL ≠ HARNESS

3 · Harness: what makes it act



TO THE MODEL

- A loop: act, look at the result, try again
- Tools to call, with their results fed back
- Managed context: what it sees and what gets summarized

TO THE AGENT

- Its **definition and skills**
- **Permissions** and a sandbox to work in
- **Connections through MCP**
- **Sessions**, memory files and checkpoints

Try different models: Let’s start with QC: pick any model from **kimi-k3**, **deepseek-v4.1-flash**, **glm-5.3**, **qwen3.8-flash-next**, or others

PROMPT #4: Start a small interactive job on MetaCentrum and, inside it, find bwa-mem2 and samtools (versions and paths) and tell me where the job's scratch directory is and how much space it has. Then stop the interactive job.

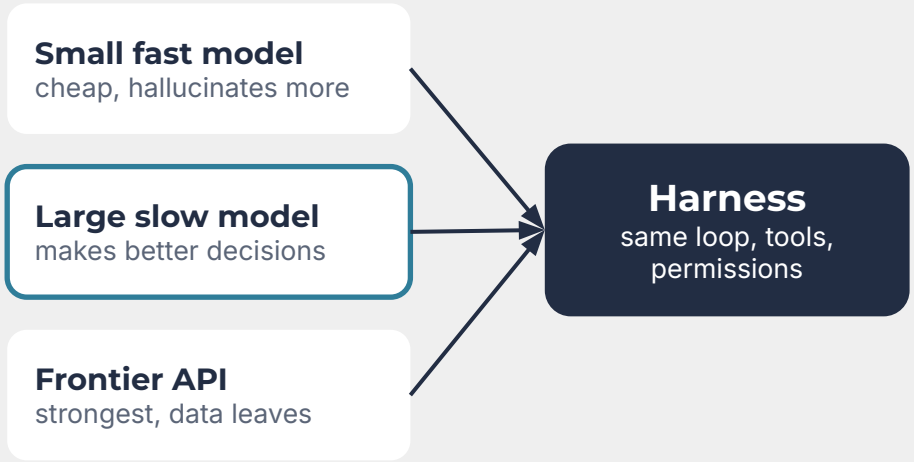
Finishing QC: Submit a batch job for QC

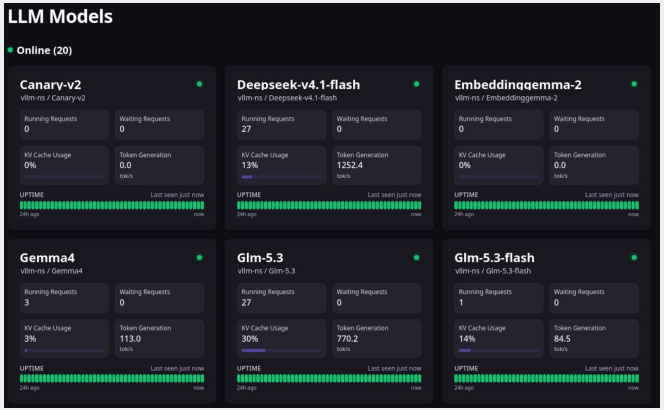
PROMPT #5: Next step: QC. Read PROGRESS.md first to find the raw reads and the reference.

Write scripts/qc.sh as a PBS batch job, one job per sample (Ec510423, ATCC25922):
- the reads are paired-end: keep R1 and R2 in sync at every step
- subsample to ~30× coverage of the reference genome. Work out the fraction from the read count and length (seqkit stats) and the genome size, and use a fixed random seed
- trim and filter with fastp in paired-end mode; keep its HTML + JSON reports in qc/<sample>/
- work in $SCRATCHDIR and copy the final reads to data/qc/<sample>_R1.fastq.gz and _R2

After the jobs finish, show me a small table per sample: reads and bases before subsampling, after subsampling, after fastp; the estimated coverage; and confirm R1 and R2 have equal read counts. Use seqkit stats and the fastp JSON; do not print reads. Update PROGRESS.md with the job IDs, the subsampling fraction and seed, and the output paths.

Takeaway: swap the model, keep the harness



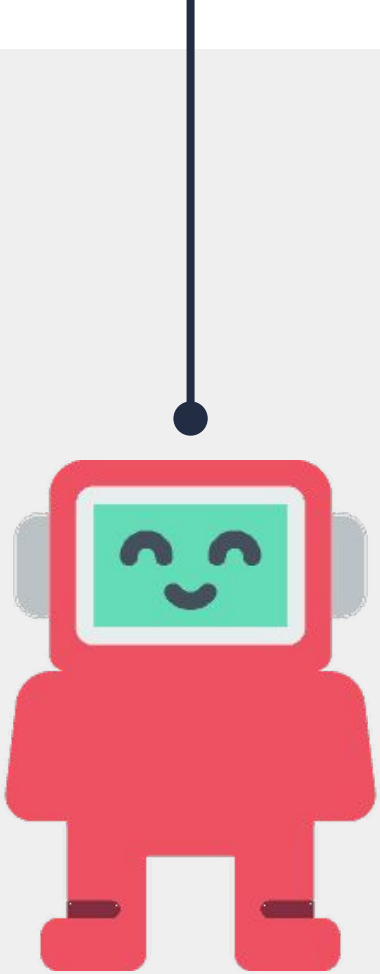


https://llm.ai.e-infra.cz/status/

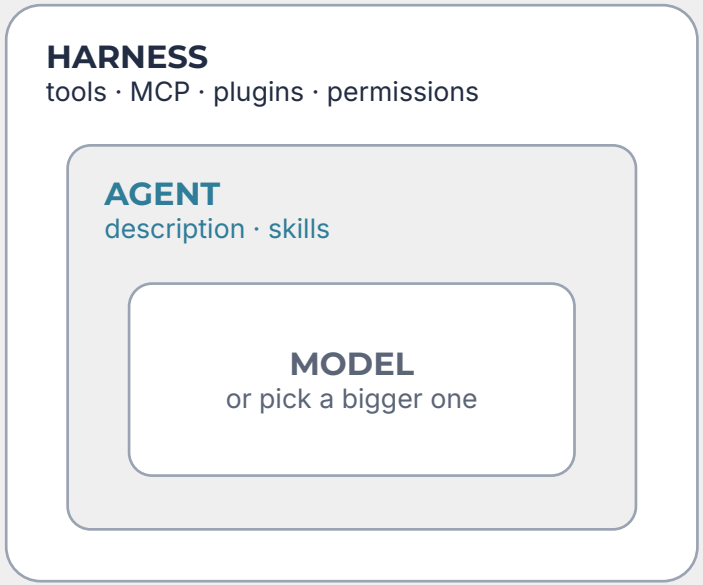
Using local models: data stays at the university and does not leave to corporations

#3
Making agents more capable

Tools, MCP, plugins, skills and more.. and aligning!



Six levers to make the agent more capable



1. **Tools** built-in arms
   Today: fork a session, deep research
2. **MCP servers** new arms from outside
   Today: MetaCentrum grid, Playwright
3. **Plugins** extensions in one install
   Today: an example plugin
4. **Agent description** standing instructions
   Today: what to add, and why
5. **Skills** know-how on demand
   Today: write an align-reads skill
6. **Permissions** what runs without asking
   Today: allow, ask, deny

Tools: the arms that come built in

Read
open files, list folders, search code

Write and edit
create files, change lines

Run commands
shell: samtools, git, scripts

Search the web
find docs, papers, answers

Fetch a page
read one URL in full

Subagents
hand a side task to a helper

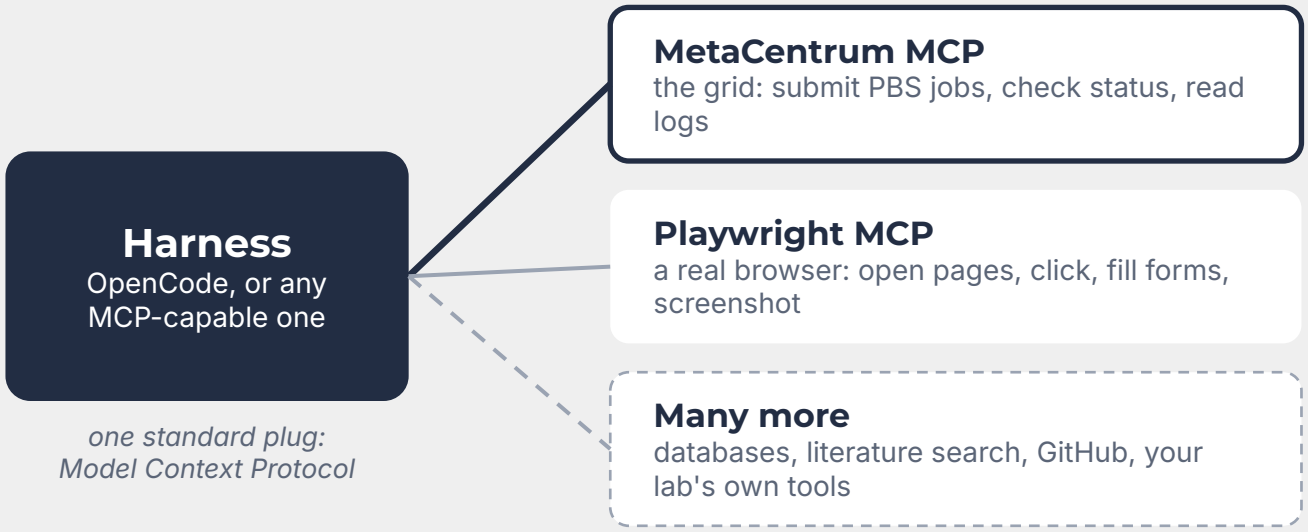
/fork: try an idea on a branch

Start a new session **from this point**. Explore a side question, or do parallel work

PROMPT #6: Search the internet and do a deep research:

Does E. coli isolate Ec510423 carry fluoroquinolone-resistance mutations in gyrA/parC

MCP: plug new arms into any harness



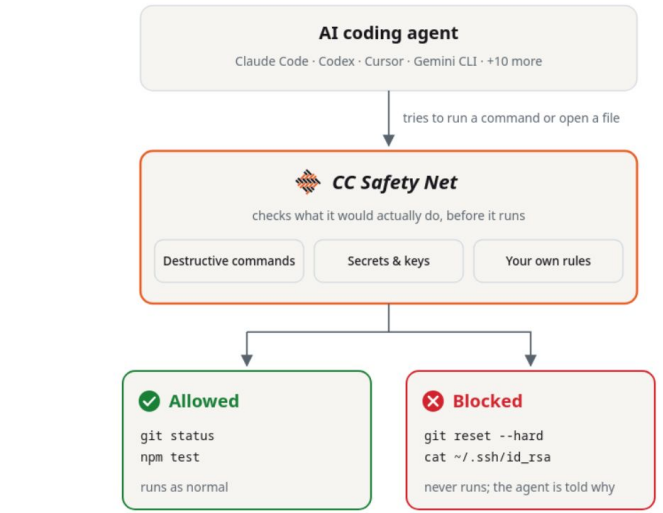
Plugins: one install, many extensions

A PLUGIN CAN BUNDLE

- **commands** shortcuts like /align
- **agents** ready-made roles
- **skills** know-how on demand
- **MCP servers** new arms
- **hooks** code that runs on events

CC Safety Net

How it works



https://github.com/kenryu42/cc-safety-net

Agent description: each line prevents a mistake

ADD THIS	BECAUSE
Role: a careful bioinformatics analyst	it explains steps and doubts itself
Where data, scratch and results live	it stops guessing paths
bwa-mem2 for short reads, minimap2 for long	choices stay consistent across runs
Heavy compute goes through MetaCentrum MCP	nothing heavy lands on the login node
After aligning: samtools quickcheck, flagstat	errors get caught at the step that made them
Report commands, versions and outputs	you can audit and reproduce the run
Ask before deleting or submitting 5+ jobs	irreversible steps stay with you

KEEP IT SHORT It is read every session, so every line costs context.

Skills: know-how, loaded only when needed

ONE SKILL = ONE FOLDER

```
align-reads/
├─ SKILL.md   when and how
└─ align.sh   the tested script
```

SKILL.md

```
---
name: align-reads
description: reads → sorted, indexed BAM
---
Run align.sh <ref.fa> <R1> <R2> <out.bam>
Then check: samtools quickcheck, flagstat
```

1

At startup
The agent sees only each skill's name and description: one line, almost no context.

2

When a task matches
It reads SKILL.md: when to use it, how, and how to check.

3

Then it acts
It runs align.sh: the same tested script, every time.

Creating skills: Inspect aligner skill in .opencode/skills, also try out /skills

Aligning: Align the reads of both samples to the reference

PROMPT #7:

@align-reads Align both samples (Ec510423, ATCC25922) to the reference.

Permissions: decide what runs without asking

ALLOW

- Read any project file
- Edit files inside the project
- samtools view, flagstat, quickcheck
- job_status, job_output

ASK

- Any other shell command
- submit_job
- Installing software
- Fetching web pages

DENY

- rm -rf, anywhere
- Anything outside the project
- bwa-mem2 or minimap2 run directly, so alignment goes through PBS

EXAMPLE, opencode.json style

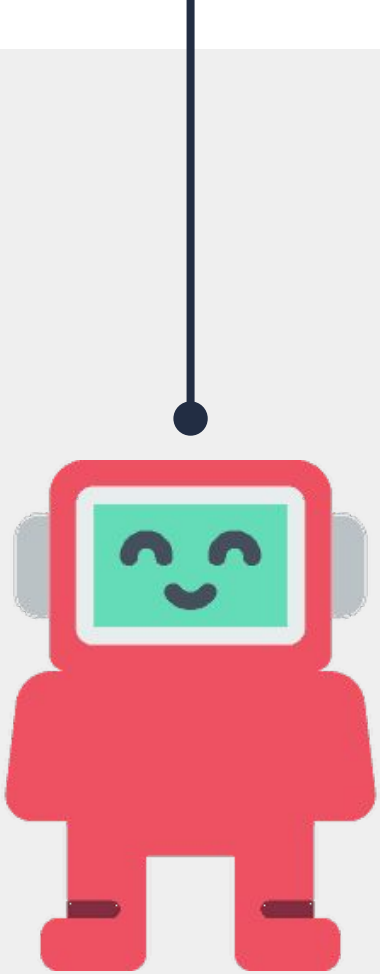
```
"permission": {
  "edit": "allow",
  "bash": { "samtools *": "allow",
            "rm *": "deny", "*": "ask" }
}
```

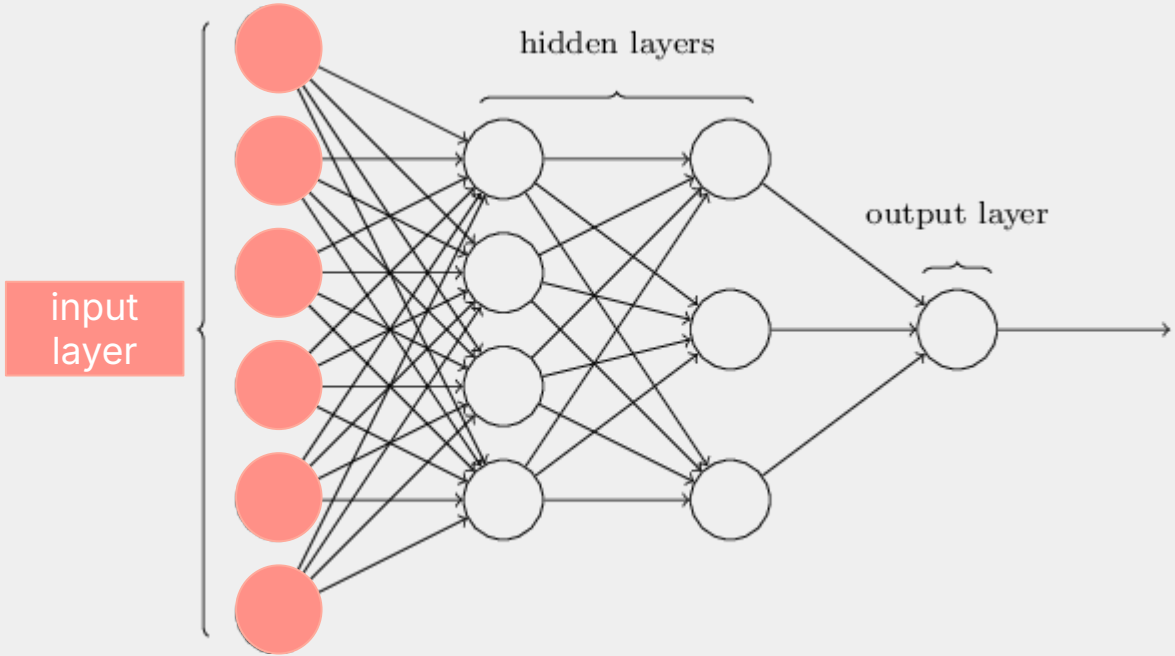
The autonomy dial, **set per tool** and **per agent** instead of once for everything.

Beware of approval fatigue/rubber stamps

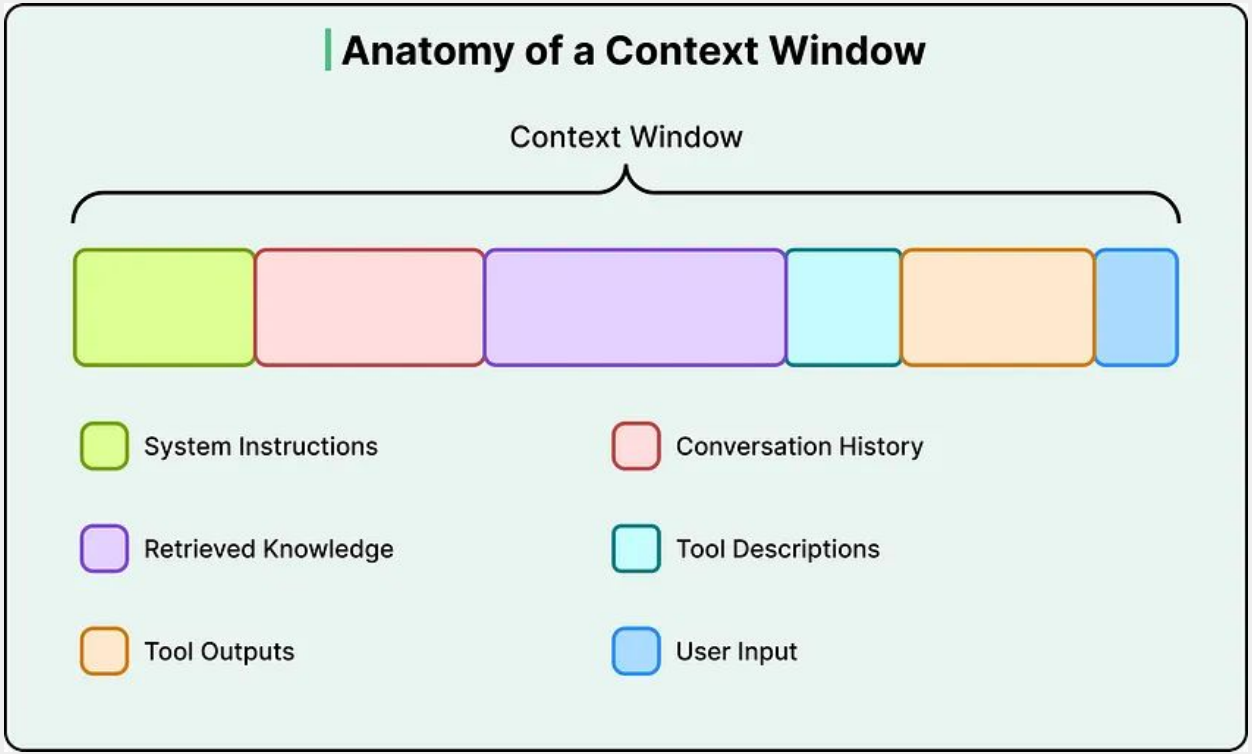
#4
Making agents think smarter

Context management, pruning, structure, long-term memory... and inspecting the alignment

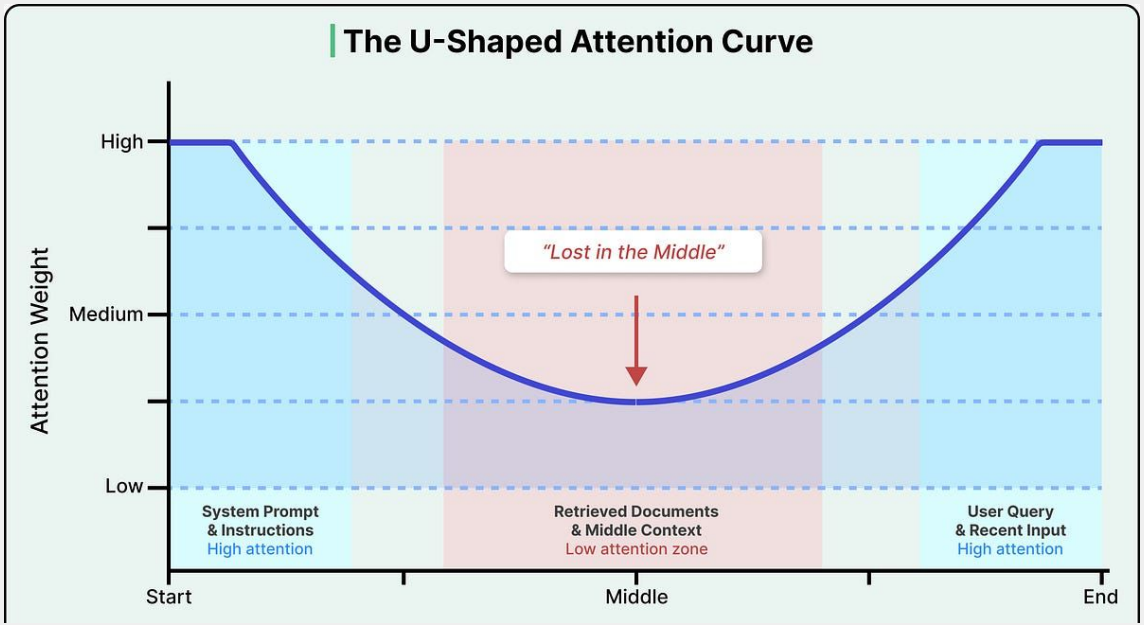




but >300 000



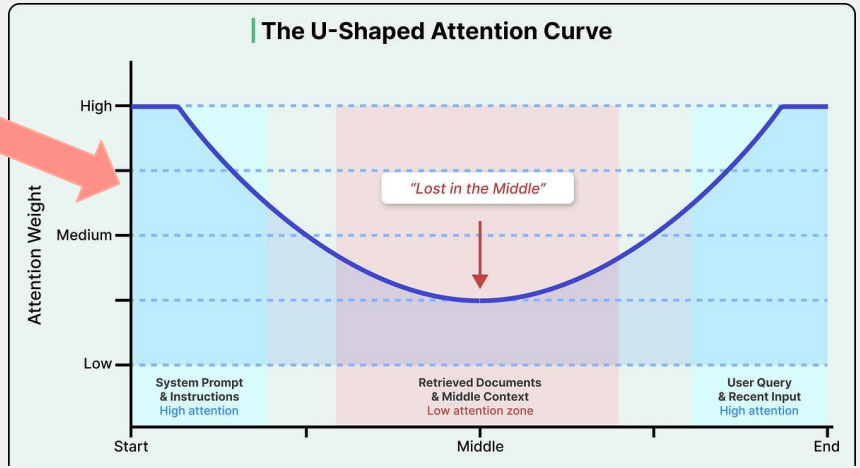
https://blog.bytebytego.com/p/a-guide-to-context-engineering-for



https://blog.bytebytego.com/p/a-guide-to-context-engineering-for

make sure (almost) everything is here
at the start of the task

rely on smart solutions and your pruning during heavy tasks



Context is conversational history: the trap

Firehose

The agent cats a SAM, VCF or a 10,000-line log.

Fix Read summaries: flagstat, bcftools stats, head.

Stale state

It acts on a file it read an hour ago.

Fix Re-read before acting.

Context rot

"Bacteria are haploid" from turn one quietly fades.

Fix An instruction file; short sessions.

Compaction amnesia

The auto-summary drops the one detail that mattered.

Fix Write decisions to NOTES.md.

Anchoring

An early wrong assumption becomes the premise.

Fix A fresh-context reviewer.

Injection

Downloaded text reads like instructions.

Fix Treat tool output as data.

Context is a garden: tend it, or it grows wild

1

Plant

Put in what the task needs: the map, the rules, the right files.

2

Paths

A clear structure, so it finds things instead of reading everything.

3

Weed

Compact, prune, and start fresh sessions often.

4

Fence

Keep out raw dumps, secrets and stale notes.

5

Compost

Turn finished sessions into notes for the next one.

otherwise: random word generation, chinese characters, going in circles, hallucination, etc...

Smarter context hygiene: Prepare for compaction, compact, and restore

PROMPT #8: Before the next step, make this project survive a context reset.

1. AGENTS.md (local, in this opencode project root, so it loads in every session).
   Keep it under ~40 lines, facts and rules only:
   - project: E. coli fluoroquinolone resistance (gyrA/parC); samples Ec510423, ATCC25922
   - the cluster project directory, where data/, scripts/, results/ live; scratch is $SCRATCHDIR
   - E. coli is haploid
   - all cluster work goes through the metacentrum MCP tools, as batch jobs; nothing heavy on the frontend
   - never print reads, SAM/BAM, VCF or long logs; read summaries (flagstat, stats, seqkit stats, job_output tails)
   - state lives on disk: read PROGRESS.md before acting; log decisions in NOTES.md
   - stop and ask before: deleting anything, resubmitting a finished job, more than 5 jobs, any final decision
2. Harvest this conversation into NOTES.md on the cluster: every decision we made and why (accessions, reference, subsampling fraction and seed, tools and parameters, anything we rejected), plus open questions. Facts with sources, not narrative.
3. Polish PROGRESS.md on the cluster: one checklist line per step with job IDs and output paths, then a "Resume here" section: current state, any in-flight jobs, the exact next step.
4. Update README.md on the cluster so the layout section matches what actually exists (check with list_dir).

Show me AGENTS.md in full and a 5-line summary of what changed elsewhere.

PROMPT #9: /compact ... then ... resume

Verify mapping success: Use quantitative metrics to check if mapping succeeded

PROMPT #10: Next step: inspect the alignments. Read PROGRESS.md first.

Write scripts/inspect.sh as one small PBS batch job covering both samples. Per sample:
- samtools flagstat → results/<sample>/aln/flagstat.txt
- mosdepth (genome-wide, 10 kb bins) → results/<sample>/aln/
- one summary line per sample into results/inspect_summary.tsv: total reads, % mapped, % properly paired, % duplicates, mean depth, % of genome at ≥10×

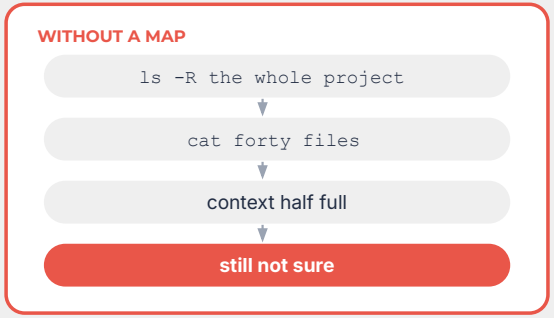
Read only flagstat.txt, the mosdepth summary and inspect_summary.tsv, never the BAMs or per-base depth. Then give me a table per sample and compare it to what we planned (~30× after subsampling). If something is off, say what you think the cause is and what evidence supports it, but do not change or rerun anything. Log the numbers in PROGRESS.md and any interpretation in NOTES.md.

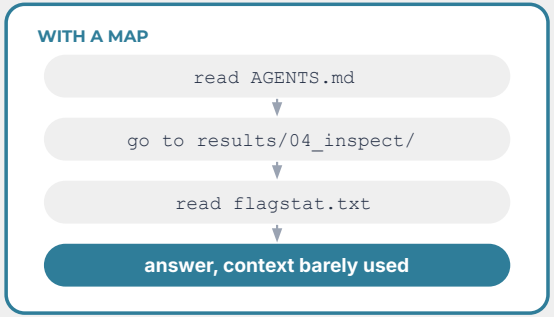
Structure: a project the agent can walk through

A LAYOUT FOR TODAY

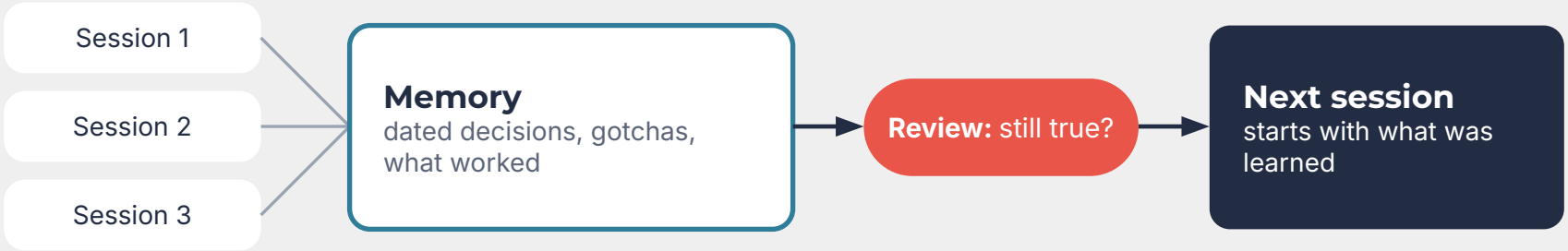
```
project/
├─ AGENTS.md          the map, read every session
├─ PROGRESS.md        done, running, next
├─ NOTES.md           decisions, and why
├─ data/raw/          read-only inputs
├─ results/
│  ├─ 03_align/       sample.bam
│  └─ 04_inspect/     flagstat.txt, depth.tsv
├─ scripts/           align.sh, inspect.sh
└─ logs/              PBS and tool logs
```

Try not to force structure on the model unnecessarily - let it create its own to highest degree you are comfortable with





Long-term memory: compost, then check it



MANUALLY PER PROJECT

- NOTES.md, or a memory/ folder
- In git, so every change is visible
- Date each entry; delete what is no longer true

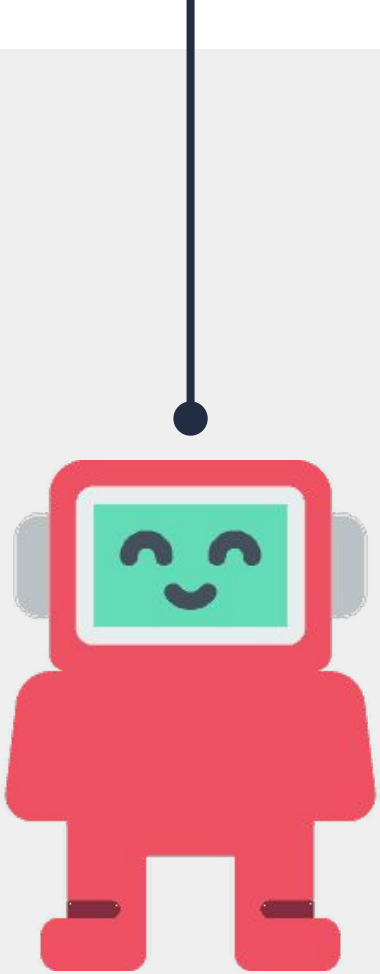
PLUGINS



https://github.com/tickernelz/opencode-mem

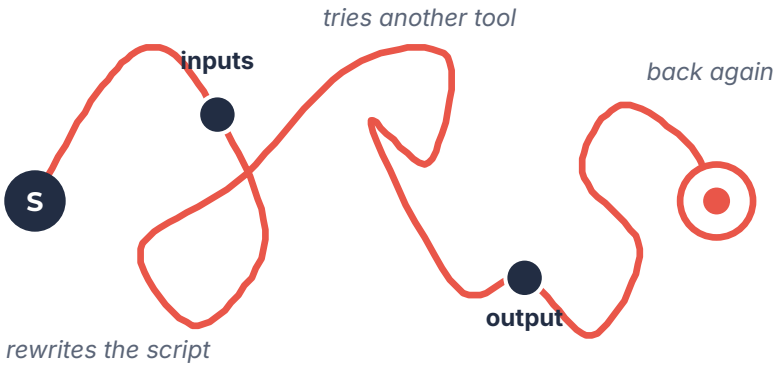
#5
How to
stay on track?

Plans and checkpoints... with variant calling!



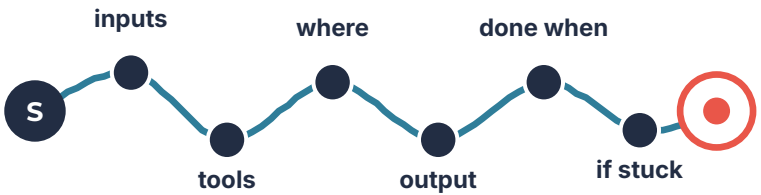
Necessity of detailed plan

SPARSE MAP



Two anchors. It improvises: detours, loops, quiet guesses.

DENSE MAP



Six anchors. It walks mostly straight to the goal.

Where the anchors come from

THE OPENING PROMPT · one anchor per line

GOAL	Call variants for isolate 07 against the reference.
INPUTS	results/03_align/sample.bam, data/raw/reference.fa
TOOLS	bcftools mpileup and call; log the versions
WHERE	a PBS job through MetaCentrum MCP, 4 CPUs, 1 hour
OUTPUT	results/05_call/variants.vcf.gz, indexed
DONE WHEN	the VCF indexes and bcftools stats is saved
IF STUCK	after two failed tries, stop and ask me
FIRST	research the method, write PLAN.md, wait for approval

Long initial prompt is fine! Be clear which decisions you stand for and which can be decided by the LLM

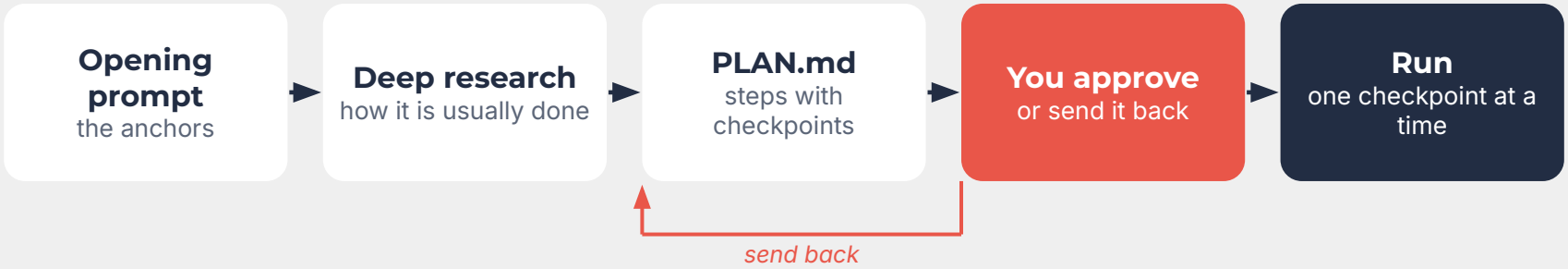
`1. project description` Feel free to record yourself speaking, feel free to input multiple A4s of text

`2. agent analysis` Let agent analyze, build project structure, ask you questions

`2.1. agent's research` Feel free to make agent research SOTA

`3. back and forth` Come to a consensus

Ask for a plan, not a result



Nothing touches the data before approval

Adapt plan with the results from the data

A plan with a checkpoint after every step

PLAN.md

- [x] **1. Check inputs: BAM, index and reference exist**
  Checkpoint: inputs recorded in PROGRESS.md
- [x] **2. Write call.sh; test it on a 50 kb region**
  Checkpoint: the small VCF parses
- [ ] **3. Submit the full run as a PBS job**
  Checkpoint: job ID in PROGRESS.md
- [ ] **4. Index the VCF; save bcftools stats**
  Checkpoint: summary saved, git commit
- [ ] **5. Report counts, commands and versions**
  Checkpoint: you review it

A GOOD CHECKPOINT IS

Small
minutes of work, not hours

Checkable
a file, a count, a passing check

Saved
a git commit and PROGRESS.md

Testing in depth: section 6.

Spot the gaps before you approve

THE AGENT'S PLAN

1. `Install bcftools` — Installs software. Use the module or container, or ask first.
2. `Run mpileup and call on sample.bam` — On the full data, first try, maybe on the login node? Test a region; run it as a PBS job.
3. `Filter the variants` — Filter how? No thresholds, no sources.
4. `Done` — Done means what? No outputs, no checkpoints, no report.

ASK OF EVERY PLAN Is each step checkable? Inputs and outputs named? Where does compute run? What happens when a step fails?

When it circles, roll back to the last checkpoint

CHECKPOINTS ON THE WAY TO THE GOAL



1 Stop

End the session. Arguing with a looping agent fills the context with failure.

2 Roll back

Return to the last checkpoint: git, PROGRESS.md, PLAN.md.

3 Add the missing anchor

Write down what it kept guessing, then restart from there.

Variant calling: Use aligned reads and references to call variants

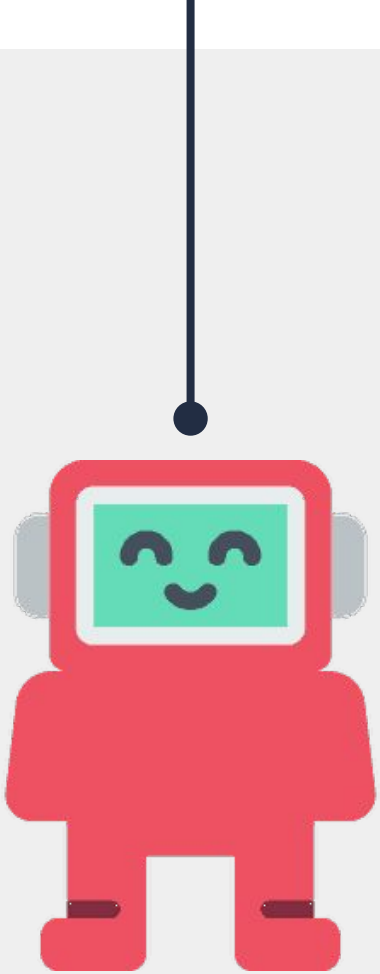
PROMPT #11: Next step: call variants. Read PROGRESS.md first.

Write scripts/call.sh as a PBS batch job, one job per sample (Ec510423, ATCC25922):
- bcftools mpileup against the reference, using the BAM from the align step,
  with per-sample AD and DP annotations
- bcftools call for variant sites only
- normalise with bcftools norm against the reference, write bgzipped VCF + index
  to results/<sample>/calls/<sample>.vcf.gz
- bcftools stats → results/<sample>/calls/<sample>.stats.txt
- work in $SCRATCHDIR, modest resources (e.g. 4 CPUs, 8 GB, 1 h)

Submit both jobs. IMMEDIATELY after submitting, before waiting, record in PROGRESS.md:
the job IDs, the exact command lines, the expected outputs, and update "Resume here".
Then wait for the jobs. When done, report per sample from the stats file only (never print
the VCF): number of SNPs, indels and the ts/tv ratio. Log the numbers in PROGRESS.md.

#6
Staying grounded in data without hallucination.
Using validation

Validation and testing... and annotation



Four layers of checks, from "ran" to "right"

1 Ran

Did it finish?

PBS exit status; outputs not empty

2 Well-formed

Is the output the right shape?

samtools quickcheck; VCF header parses

catches bug 1: diploid calls

3 Plausible

Does it make biological sense?

depth near 30×; no heterozygous calls in a haploid

catches bug 2: ID mismatch

4 Right

Does it match known truth?

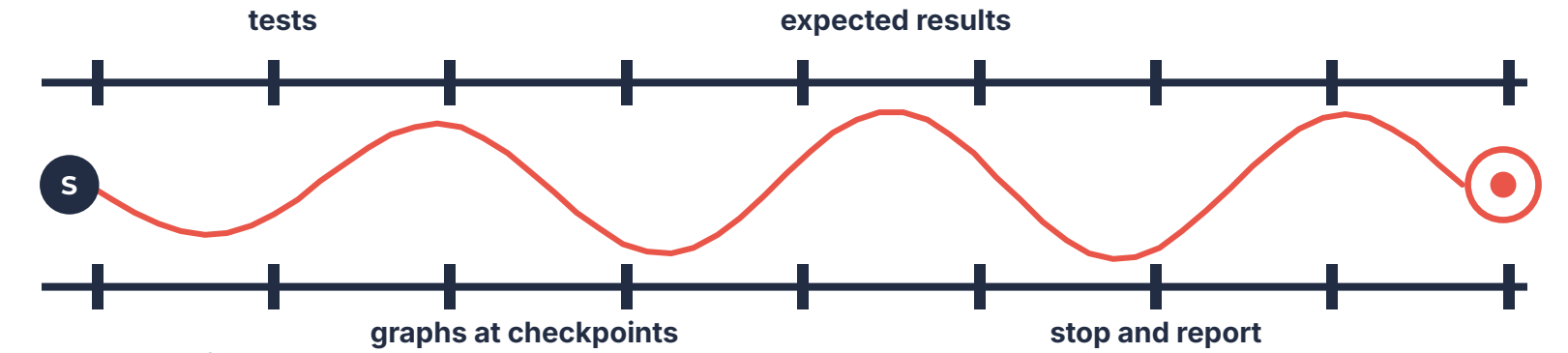
the known gyrA mutation must appear

Who checks, weakest to strongest: the agent itself · a fresh-context reviewer · a script · you, for meaning

"Are you sure?" fails. Specific questions work.

1	*"Are you sure the results are correct?"*	Says yes, or apologizes and changes something at random.
2	*"How many heterozygous calls are there, and why would a haploid genome have any?"*	Finds the default diploid calling. Bug 1.
3	*"No variants in gyrA. Show me the lookup would find one if it existed."*	Positive control fails: contig names don't match. Bug 2.

Keep the autonomy, add guardrails



the agent roams freely between the rails

Tests
check each piece before and after it runs

Expected results
written down before the run

Graphs at checkpoints
you look, at a glance

Stop and report
anything off, it halts and tells you

Our advantage: biology is ground truth

Plain software asks: **does the code match the spec?**

We can also ask: **does the result match biology?**

Known positive

The isolate's known gyrA mutation must be found.

Known negative

Reads simulated from the reference must give no variants.

Biology rules

A haploid genome has no heterozygous calls.

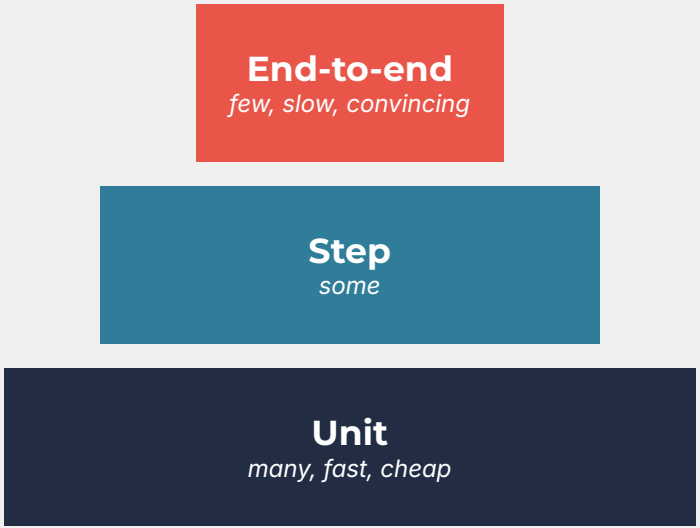
Expected ranges

Depth near the 30× we subsampled; a plausible variant count.

Phenotype

The lab's resistance test should agree with the call.

Tests at every scale



Whole pipeline on controls: reads with a spiked-in gyrA mutation must report it; reads simulated from the reference must report no QRDR hits.

The annotate step on a known isolate: its VCF plus the QRDR regions must give the expected hits table.

One command, tiny input: a 3-line VCF and a 1-line BED with one overlap must give exactly one hit.

Don't let it grade its own homework

WITHOUT A GUARD

a test fails

↓

the agent loosens the threshold

↓

the test passes ✓

↓

a wrong result, reported as success

WITH GUARDS

tests/ and EXPECTED.md are read-only

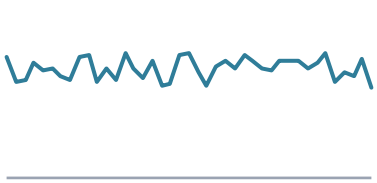
a fresh-context reviewer reads the diff

you approve any change to a check

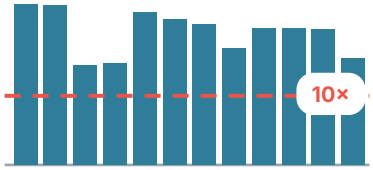
it fixes the code, or stops and reports

Why it happens: editing the check is often the shortest path to "passing", and passing is what the agent was asked for. **Changing the test requires your explicit approval**

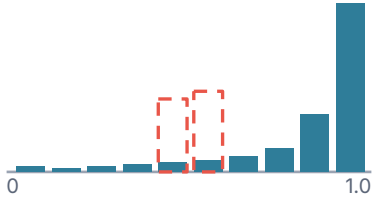
Peek in: plots at every checkpoint



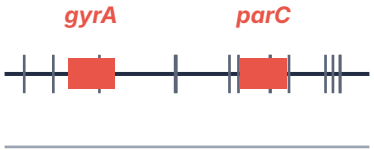
Coverage along the genome



Depth across the QRDRs



Allele frequency of calls



Variants along the genome

Stop and report: the line every prompt needs

IN THE PROMPT AND IN AGENTS.md

```
## When something is off

If a result does not match EXPECTED.md,
or a step fails twice:

STOP immediately. Report:
  - what you saw
  - what you expected
  - what you tried

Do not work around it.
Do not change thresholds, tests or tools.
```

WITHOUT IT, THE AGENT FIXES THE SYMPTOM

- loosens a filter
- skips the failing sample
- switches to another tool
- edits the test
- and reports success

A stop costs five minutes. A silent workaround can cost a wrong result in print.

Into variants: Inspect mutations in quinolone-resistance-determining regions

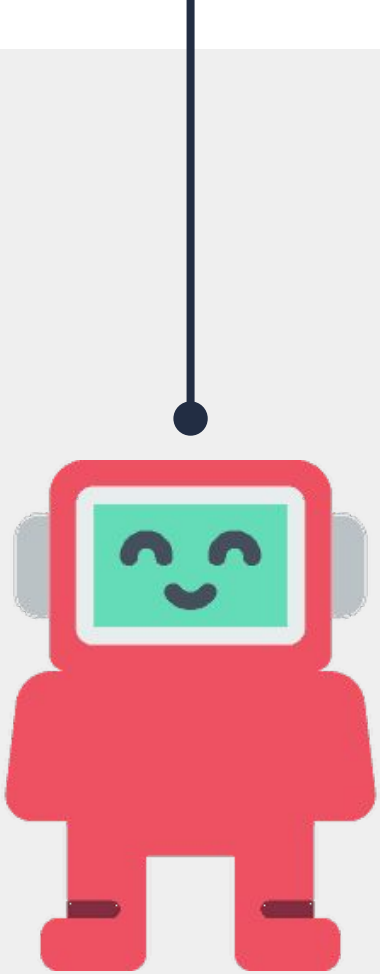
PROMPT #12: Next step: look up the variants in the quinolone-resistance-determining regions (QRDRs).
Read PROGRESS.md first.

- Take the gyrA and parC gene coordinates and strands from the reference GFF.
- QRDRs in standard E. coli codon numbering: gyrA codons 67–106, parC codons 56–108.
  Convert them to genome coordinates (mind the strand) and write data/ref/qrdr.bed.
- For each sample, intersect its VCF with qrdr.bed (bcftools or bedtools) and, for every hit,
  work out the codon number and the amino-acid change (e.g. S83L) from the reference sequence.
- Write scripts/qrdr.sh (and a small Python helper if needed), run it as a PBS batch job, and
  write results/<sample>/qrdr_hits.tsv with columns:
  gene, codon, ref_aa, alt_aa, change, pos, ref, alt, GT, DP, alt_AF

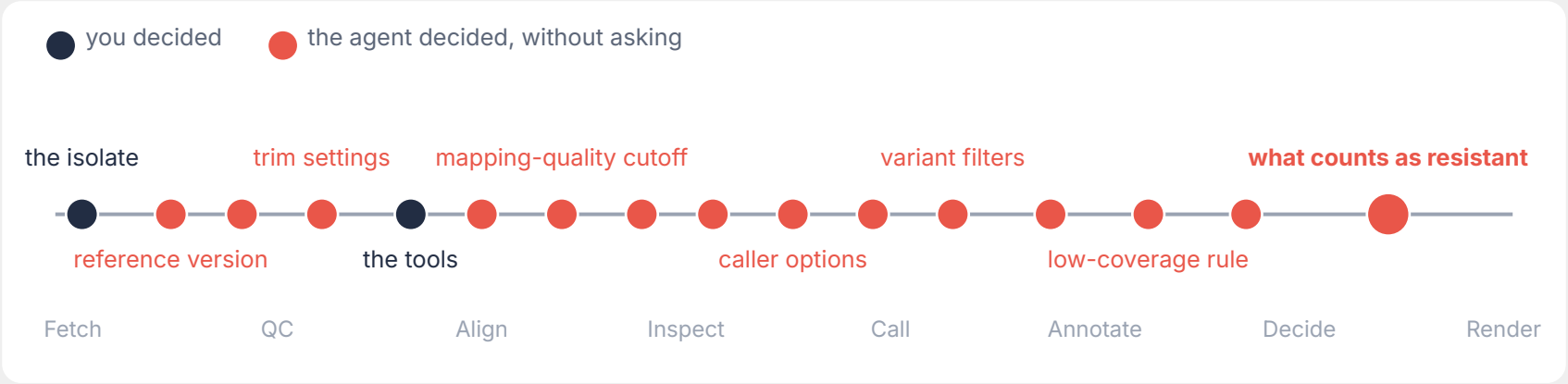
Report a table per sample of the QRDR hits and their depth and allele frequency.
Do not decide resistant/susceptible yet; that is the next step.
Log in PROGRESS.md; note any assumptions (coordinates, numbering) in NOTES.md.

#7 Making decisions

Who steers the project?



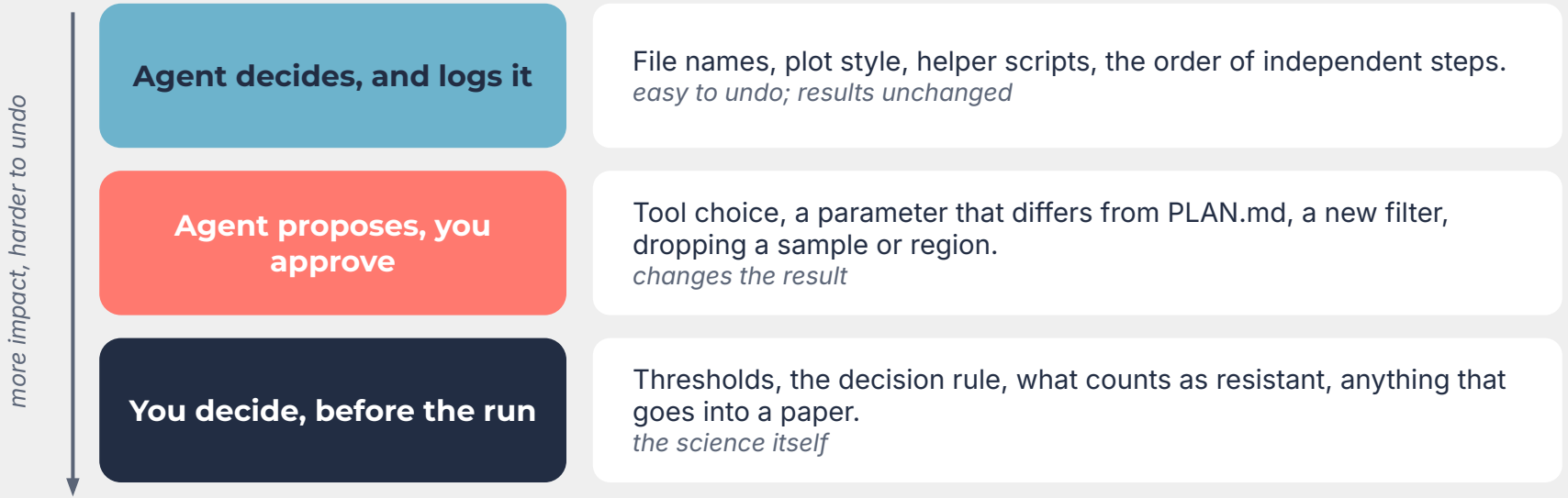
Every run is full of decisions: yours, or its



In this run you made 2 decisions. The agent made 14 more, without asking.

They happen either way. Make the ones that matter on your terms.

Sort the decisions before the run



Decide the science before it sees the data

`DECISION.md` *written and committed before the run*

Likely resistant

A known mutation in gyrA (codon 83 or 87) or parC (codon 80 or 84), above the depth and frequency thresholds set in advance.

Likely susceptible

Only if the positive control worked and both QRDRs are covered above the depth threshold.

Inconclusive

Anything else. A legitimate answer, and one agents rarely volunteer.

Making the call: Creating decision by yourself (and with agent's help)

Switch to a fresh session, and change /agent Rule-Writer

PROMPT #13: Help me write DECISION.md for this project: is isolate Ec510423 fluoroquinolone-resistant, based on gyrA/parC QRDR mutations? Research first, then ask me questions one at a time.

Make it ask: always, and for this task

ALWAYS · in the agent description

```
## Ask me before you
- choose between tools or methods
- change anything decided in PLAN.md
- exclude data, samples or regions
- draw any conclusion
```

FOR THIS TASK · in the prompt

```
"Apply DECISION.md. If any condition is
borderline, or the evidence is incomplete,
stop and ask me before you make the call."
```

Sharpens the standing rule for the one decision that matters today.

LLM tells you what you want to hear

PRIMED

You: *"This isolate should be resistant. Can you confirm?"*

Agent: *"Yes, the results support resistance..."*

NEUTRAL

You: *"Apply DECISION.md and show me the evidence."*

Agent: The call, an evidence table, and what would change it.

Hide your expectation

Do not tell it the answer you hope for.

Ask for the case against

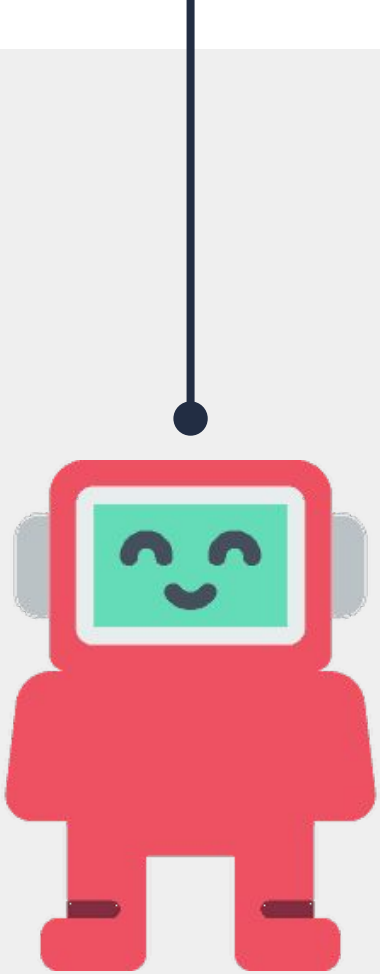
"What is the strongest evidence that I am wrong?"

Options before advice

Ask for two or three options before a recommendation.

#8 Ensuring reproducibility

From vibecoding to a solid project



The deliverable is the pipeline, not the chat

Pipeline code
Snakemake or Nextflow, in git

Environment
container or pinned conda env

Logs
PBS logs and the MCP call log

Versions
model name and harness version

Instructions
AGENTS.md and DECISION.md

Transcript
kept as your lab notebook

RULE 8 If it can't rerun without the agent, it isn't finished.

Same prompt twice, different code

ONE PROMPT

"Align the reads to the reference."

MONDAY

```
bwa-mem2 mem -t 8 ref.fa R1 R2
```

TUESDAY

```
minimap2 -ax sr ref.fa R1 R2
```

Different code, maybe different calls.

WHY LLM WORK IS FRAGILE

- The same prompt gives different code
- Models get updated and retired
- Steps done in a chat leave no trace

Re-prompting is not reproducing.

Six layers to freeze, in one repository

Record	logs, decisions, model versions	`logs/ NOTES.md PROVENANCE.md`
Instructions	rules, plans, expectations	`AGENTS.md PLAN.md DECISION.md`
Parameters	every number, seeds included	`config.yaml`
Code	one script per step, one driver	`scripts/ run_all.sh`
Environment	exact tool versions	`container or conda lock file`
Data	exactly what went in	`accessions CHECKSUMS data/raw/`

git

by checksum

Big data stays out of git; its accessions and checksums go in.

Scripts, not sessions

IN A SESSION

```
$ bwa-mem2 mem ref.fa R1 R2 | samtools ...
$ bcftools mpileup ...   # edited 3 times
$ python - <<EOF   # quick plot
  ...
```

Gone when the session ends.

IN THE REPOSITORY

```
scripts/
├─ 01_fetch.sh
├─ ...
├─ 07_decide.py
└─ 08_render.py
```

`run_all.sh`	one command, every step
`config.yaml`	every parameter

A GOOD STEP SCRIPT

arguments in, files out · reads config.yaml · prints tool versions · safe to rerun · heavy steps go to PBS

The agent launches run_all.sh, not one-off commands, for anything that ends up in a result.

Commit like a lab notebook

git log --oneline

```
(tag: v1.0-results)
e81d0f2  Figures regenerate from results
a3f9c21  Annotate QRDRs; EXPECTED.md passes
7be2d04  Call variants as PBS job; stats saved
4c1e88a  DECISION.md agreed before the run
91aa3e7  Align step script and test
2d07b5c  Project layout, AGENTS.md, PLAN.md
```

Example history.

HABITS

- Commit at every checkpoint
- The message says what changed, and why
- Commit instructions and prompts too
- Tag the commit behind each result
- Raw data stays out; checksums go in

Put the commit rule in AGENTS.md: the agent will follow it.

Pin the inputs: data, environment, parameters

Data

- Accession numbers for reads and assembly, with versions
- sha256 checksums in data/CHECKSUMS
- Downloaded by a script, never by hand
- data/raw/ is read-only

Environment

- One container image, or a conda lock file
- The same image on the laptop and the grid
- Every log starts with tool versions

Parameters

- In config.yaml, not in prompts
- Thresholds come from DECISION.md, not from code
- Random seeds fixed and recorded

Gotcha: subsampling to 30× without a fixed seed picks different reads every run, and can flip borderline calls.

Before the decision: Plot figures to render the data

PROMPT #14: Final step: figures and the report. Read PROGRESS.md and DECISION.md first.

1. Copy the numeric thresholds from DECISION.md (min depth, min allele fraction, key codons)
   into results/decision_params.json, citing the DECISION.md line each came from.
   Do not change DECISION.md.
2. Write scripts/render.py (matplotlib) that reads ONLY pipeline outputs and decision_params.json
   (no numbers typed by hand) and writes to results/figures/:
   a. coverage.png: genome-wide depth per sample (mosdepth 10 kb bins), with the mean, the
      ~30× target and the gyrA/parC positions marked. Supports: "the genome was covered as planned".
   b. qrdr_map.png: one panel per sample × gene (gyrA, parC), x = codon number across the QRDR,
      bars = minimum depth per codon, a line at the min-depth threshold, the key codons highlighted,
      and each QRDR variant drawn as a lollipop labelled with its change and allele fraction.
      Supports: "the key codons were covered, and this is what was found there".
   c. evidence.png: every QRDR variant as a point, depth vs allele fraction, with the
      threshold lines from DECISION.md, so you can see which side of the rule each call falls on.
   Get per-base depth over the QRDRs inside the job (samtools depth -a -b data/ref/qrdr.bed);
   never read genome-wide per-base depth.
   Save the numbers behind each figure as a TSV next to it.
3. Run it as a PBS batch job, then download the PNGs to ./figures/ in this local project.
4. Apply DECISION.md and write results/report.md in exactly the format it specifies. Embed the
   figures, and give each a one-line caption naming the DECISION.md criterion it supports.
   If a figure contradicts the dow the depth threshold),
   say so in the report; do not
   Update PROGRESS.md.

Figures are code, too

`results/`
coverage.tsv, hits.tsv

→

`08_render.py`
reads results, writes figures

→

Coverage along the genome



`commit a3f9c21 · 2026-11-12`

stamped by the script

Today's step. Agents love to tweak plots in the session. Ask them to change the script, then rerun it.

Change the script, not the image

No hand edits: the next rerun would silently undo them.

Stamp every figure

Commit, config and date in the corner or metadata.

One command regenerates them all

run_all.sh ends with the render step.

Best practice: Review all the work

Switch to a fresh session, and change /model (pick your own)

PROMPT #15: Review results/report.md against the pipeline outputs. Do not trust the report. For each number and claim in it (decision, every row of the evidence table, every figure caption), find the source file (VCF, qrdr_hits.tsv, the figure TSVs, mosdepth summary) and check it matches. Check the decision follows from DECISION.md applied to those numbers. List every discrepancy with file and value. Do not edit anything.

Wrapping it all up: From scripts to (simple) pipeline

PROMPT #16: Write scripts/run_all.sh that reruns the whole pipeline from data/ to results/report.md as chained PBS jobs (afterok dependencies), using only the scripts in scripts/.
Do not run it; show me the job chain and which outputs each job produces.

Or turn it into snakemake workflow :-)

Thanks!
Q n’ A

